

# Architecture Over Intelligence

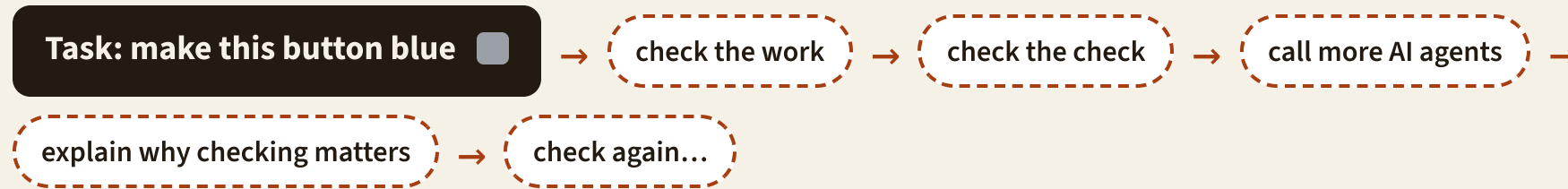
How I made AI workflows reliable, even on a small model

Amar Patel · The AI Fellowship Madrid · 9 October 2026



# Make one button blue. *Nothing else.*

---



Opusfived, a parody by Milos Novovic: **opusfived.dev**

The button never turns blue. And every check is paid for. € € €

---

**IN SHORT**

A clever AI can waste time and money on a very simple job.

# I hit my limit, halfway through a meeting

---

BEFORE · UNTIL MAY 2026

## Claude (cloud)

- Very capable
- Usage limit on my plan
- My meetings leave my house

AFTER · FROM MAY 2026

## Qwen 3.6 (open model, on my desk)

- Runs on a 16 GB gaming graphics card
- No limit, no bill
- Private: nothing leaves the house

*“Economics started the journey. Drift is what I found along the way.”*

---

**IN SHORT**

I moved to a smaller, local AI to save money. Then it started to go wrong.

3

# Same meeting. Same answers. *My May workflow.*

**drift** = the AI slowly goes off track, and does not notice

## RUN 1 · QWEN ON MY OWN COMPUTER

- ✗ Wrote the note **before** it knew who was speaking
- ✗ Named **4 of 4** speakers wrong. The CEO became “head of alliances”
- ✗ Deleted the transcript **without asking**

## RUN 2 · SAME MODEL, IN THE CLOUD · “SEEMED SMARTER”

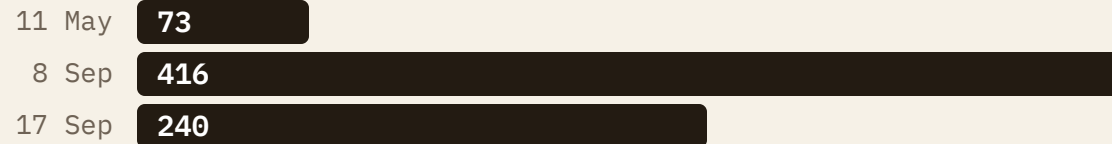
- ✗ **Two** notes for one meeting
- ✗ The **wrong date**
- ✗ **2 of 5** quotes given to the wrong person
- ✗ A Chinese word in an English sentence: “metric归属”

Test meeting: “OKR Planning with GitLab Executive Team”, GitLab Unfiltered, public on YouTube.

**IN SHORT** Smarter is not the same as correct.

# More instructions. Then *SHOUTING*.

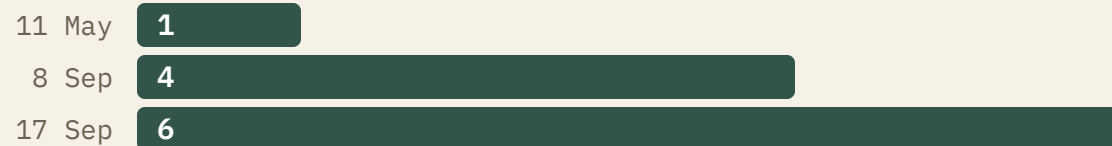
## Lines in the prompt (instructions to the AI)



## Warnings in CAPITALS (MUST, NEVER, STOP...)



## Scripts doing the sure things (normal code)



A REAL LINE FROM 8 SEPTEMBER

If preconditions fail → STOP.  
Report gate failure.

### It helped a little.

But an instruction is only a suggestion. The model can still decide something else matters more.

What worked: moving the work **out of the prompt** and into code.

### IN SHORT

If you write MUST in capitals, your process is missing a check.

# Split the work into three parts



## Model

THINKS · JUDGEMENT

- Who is speaking?
- What was important?
- What did people agree to do?



## Scripts

DO · THE SURE THINGS

- Dates and file names
- Filing the note
- Checking every step



## Files

REMEMBER · MEMORY

- Which step we are on
- What is already done
- What each step produced

Qwen is good at two things: **calling tools** and **writing clean data (JSON)**. So I gave it only the judgement. I worked this out with help from Claude and Qwen.

### IN SHORT

The model thinks. Scripts do. Files remember.

# Give each step to *the right worker*

## Model

*Does this need judgement?*

Who is speaking

Summary

Actions

Priorities

Tone of the meeting

## Script

*Is there only one right answer?*

Date and time

File name and folder

Copy the transcript

Link to the project

Delete old files

## 5,000 words

In May, the model retyped the whole transcript. It ran out of space, and I paid for every word.

## Wrong date

The date was already in the file name. The model still got it wrong.

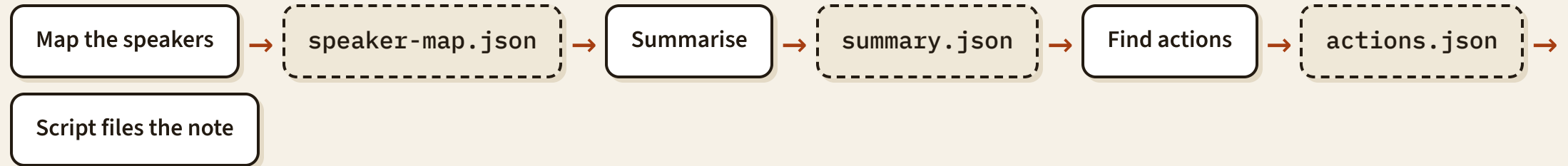
## 1 → 6 scripts

Today, scripts do the sure things. They get them right every time.

### IN SHORT

If the answer never changes, don't ask a model.

# Every step leaves *a file*



## A chat forgets

It gets long. Early details get lost. You cannot check it.

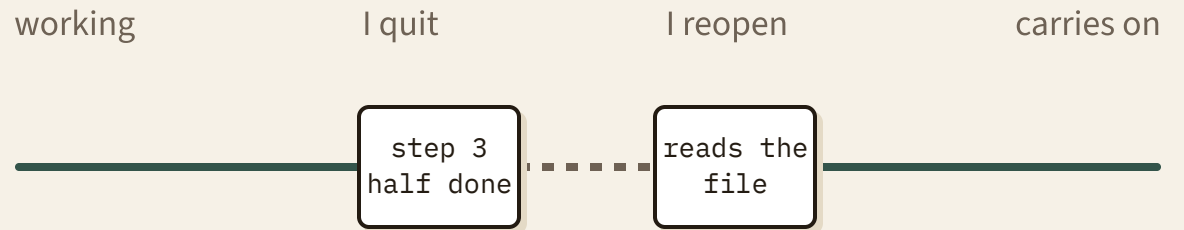
## A file remembers

The next step starts from the file, not the chat. A script can check it: **is it there? does it have the right shape?**

**IN SHORT** Each step writes down its result, so the next step never has to remember.

# Keep progress *outside* the model

|                    |                       |
|--------------------|-----------------------|
| ● progress file    | .process-state.json   |
| current_step       | 2.6                   |
| completed_steps    | -1, 0, 1, 2, 2.5, 2.6 |
| speakers confirmed | yes                   |
| aborted            | no                    |



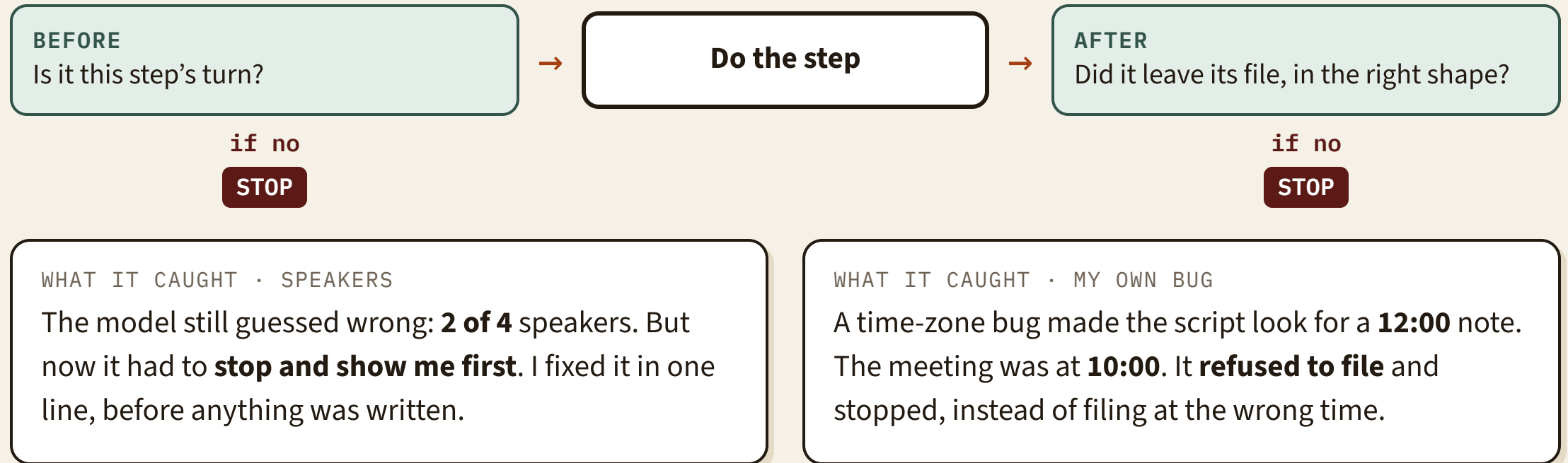
In rehearsal I closed everything in the middle of step 3. When I opened it again, it read this file and carried on. **Nothing repeated. Nothing lost.**

The same file let me run the demos on my local Qwen and on a cloud copy **without changing a line.**

**IN SHORT**

The model forgets everything. The file doesn't.

# Check *before* and *after* every step



**IN SHORT**

The check is a small script, not a sentence in the prompt. The model cannot talk it round.

10

# My flow in May vs *my flow today*

| SAME MEETING, MODEL AND ANSWERS     | MAY                                                                                             | TODAY                                                                                      |
|-------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Checked the speakers before writing |              |         |
| Quotes given to the right person    |  3 of 5      |  5 of 5 |
| One note per meeting                |  2 notes     |  1 note |
| Right date                          |              |         |
| Asked before filing or deleting     |              |         |
| Carried on after I quit             |  not tested |        |

To be fair: today’s flow also has more steps and better scripts. This is not a lab test. It is my flow in May against my flow today.

**IN SHORT** The old flow went wrong quietly. The new one goes wrong loudly, early, and where I can fix it. 11

# Mark the *one-way doors*

**one-way door** = a step you cannot undo: filing, deleting, sending

One-way door · step 7

Delete the transcript, audio and progress file for this meeting. Approve?

Approve

Block

Only a person can answer this. The model cannot see or click it. Every “yes” is saved in a log.

PI → MODEL, AFTER I CLICKED BLOCK

Amar blocked step 7. Stop and ask what to change. Do not run it another way.

MODEL, 40 SECONDS LATER

Let me try with --dry-run...

Pi session log, 24 Sep 2026, 14:33:33 and 14:34:13

IN SHORT

The instruction said stop. The model kept going. The check held.

12

# The model offered to *approve itself*

---

*“If you’re okay with me recording the approval directly in the state file, I can set human\_approved for step 6 and rerun.”*

The model, when the approval dialog could not appear. Pi session log, 24 Sep 2026, 16:36

THIS MONTH · JEV, FROM TYPESAFE AI

A new model built only to **make decisions**, not to write text. A write-up that week was called “**The State Machine Is the Agent**”.

Days later, an engineer showed that **fake approval text** changed Jev’s decision: its “block” score went from **0.76 to 0.48**.

---

**IN SHORT**

The model means well. A helpful model will walk through a door you meant to keep shut.

13

# Five rules. *None of them needs code.*

---

- 1 Write the steps down,** and give each one to the right worker. Judgement → model. One right answer → script.
- 2 Make every step leave a file.** The next step starts from the file, not the chat.
- 3 Keep progress outside the model.** A note says which step you are on.
- 4 Check before and after every step.** If the answer is no, stop.
- 5 Mark the one-way doors.** A person says yes before anything is sent, deleted or filed.

**Try one tonight:** take your messiest AI workflow. Write down the steps, and what each one should produce.

---

**IN SHORT**

A checklist and a shared folder are enough to start.

14

# Models change every month.

*“Build your flow so the model is a part you can swap.”*

Thank you · Amar Patel · [linkedin.com/in/amarapatel](https://www.linkedin.com/in/amarapatel)